

1. Génération des données brutes (data.py)

Pour simuler un environnement réel avec des anomalies, vous avez utilisé un script Python capable de générer **7000 enregistrements** réalistes pour le secteur des télécoms tunisiens.

- **Réalisme local** : Utilisation de listes de noms tunisiens (ex: *الدريدي, بن عرفة*) et de villes/quartiers spécifiques (*بنزرت, بوحجلة, طبرقة*).
- **Scénarios métier** : Création de modèles de conversations couvrant divers problèmes (réseau, facturation, internet, offre 4G).
- **Injection volontaire d'anomalies (Data Quality)** : Pour tester la robustesse de votre futur processus ETL, le script a volontairement introduit :
 - **IDs manquants** (valeurs None ou NULL).
 - **Dates incohérentes** (mélange de formats : 2025.12.31, 01/02/25, Feb 10, 2025).
 - **Doublons** (répétition de certaines lignes).
 - **Casse hétérogène** (ex: "TUNIS" vs "tunis").

2. Exportation Initiale (telecom_raw_data_7000.json)

Le résultat du script est un fichier JSON brut contenant les colonnes suivantes :

- comment_id, user_name, intent, entity_location, entity_neighborhood, entity_service, input_text, output_text, action_required, sentiment, timestamp, status.

3. Migration et Encodage (Guide Complet utf16.docx)

C'est une étape cruciale pour la gestion de la langue arabe :

- **Le problème** : Les fichiers JSON sont nativement en **UTF-8**, mais SQL Server (SSMS) interprète mal ce format lors de l'importation massive, ce qui transforme l'arabe en caractères illisibles (???).
- **La solution** : Utilisation d'une commande **PowerShell** pour convertir le fichier `telecom_raw_data_7000.json` (UTF-8) en un nouveau fichier `final_data_utf16.json` (encodage Unicode/UTF-16 LE).
- **Résultat** : Les caractères arabes et les emojis sont parfaitement préservés pour l'intégration dans la base de données.

Résumé du flux : Script Python → JSON (UTF-8) → Conversion PowerShell → `final_data_utf16.json` (Prêt pour l'ETL)

1. Phase E : Extraction et Ingestion (SSMS)

L'objectif était d'intégrer les données brutes dans un environnement SQL.

- **Source** : final_data_utf16.json (JSON brut).
- **Action** : Création de la base telecom_db et de la table final_data_table.
- **Colonnes importées** : comment_id, user_name, intent, entity_location, entity_neighborhood, entity_service, input_text, output_text, action_required, sentiment, comment_timestamp, status.

2. Phase T : Transformation et Nettoyage (SSMS)

Cette phase a servi à fiabiliser et à localiser (arabe) les données.

- **Nettoyage** : Suppression des doublons et gestion des valeurs NULL (IDs générés à partir de 7000).
- **Standardisation** : Conversion des formats de dates en un type DATE propre (comment_date_clean).
- **Traduction** : Les valeurs des colonnes intent, action_required et sentiment ont été traduites en arabe.
- **Renommage des colonnes** : Pour une meilleure compréhension métier, les colonnes sont devenues :
 - id, اسم_الحريف, المشكل, الولاية, البلاصة, الخدمة, الشكاية,
 - الجواب, اش_لازم_يتعمل, الشعور, تاريخ.

3. Phase L : Chargement et Data Warehouse (SSMS)

Passage d'une table plate à un modèle en étoile dans telecom_DWH.

- **Architecture :**

1. Les Tables de Dimensions (Les axes d'analyse)

Chaque dimension contient des données descriptives qui permettent de "filtrer" ou de "regrouper" les réclamations.

Dim_Client : Contient l'identité de l'abonné.

- **Colonnes** : id_client (Clé primaire), Nom_Client (اسم_الحريف).

Dim_Geographie : Permet une analyse spatiale des problèmes.

- **Colonnes :** id_geo (PK), Wilaya (, (الولاية) Delegation (البلاصة)).

Dim_Probleme : Définit la nature technique de la plainte.

- **Colonnes :** id_prob (PK), Probleme (, (المشكل) Service_Type (, (الخدمة) Action_Requise (اش_لازم_يتعمل).

Dim_Temps : Permet de suivre l'évolution temporelle (par jour, mois, année).

- **Colonnes :** id_date (PK), Date_Full (تاريخ).

Dim_Description : Stocke les contenus textuels longs (pour ne pas alourdir la table de faits).

- **Colonnes :** id_desc (PK), Texte_Plainte (, (الشكاية) Texte_Reponse (الجواب).

Dim_Sentiment : Pour l'analyse de la satisfaction client.

- **Colonnes :** id_sent (PK), Label (الشعور : إيجابي، سلبي، محايد).

• 2. La Table de Faits : Fact_Reclamations

C'est le cœur de votre architecture. Elle ne contient presque que des chiffres et des clés étrangères.

Clés Étrangères (FK) : Elle pointe vers chaque dimension citée plus haut (id_client, id_geo, id_prob, etc.). Cela permet de relier, par exemple, une plainte spécifique à un client précis et à une région précise.

Mesure (Metric) : La colonne **Nombre_Rec1** (fixée par défaut à 1). C'est ce que vous allez "sommer" ou "compter" dans vos rapports Power BI ou Excel pour connaître le volume de plaintes.

Clé Primaire : id_fact (qui correspond à l'ID technique récupéré depuis votre phase de transformation).

- **Préparation Chatbot :** Création de la vue **View_Export_Chatbot** sélectionnant uniquement les colonnes pertinentes.
- **Export :** Génération d'un fichier intermédiaire dataset.json.

4. Phase Big Data (Hadoop / Hive)

Passage à l'échelle supérieure pour simuler un environnement de production massif.

- **Stockage (HDFS) :** Le fichier a été uploadé sur le cluster HDFS dans /user/admin/pfe/data/.

- **Traitement (Hive) :** Création d'une base telecom_bigdata et d'une table externe reclamations_hive.
- **Colonnes Hive :** instruction, response, sentiment, category, location.
- **Analyse :** Exécution de requêtes de statistiques (comptage par catégorie et par région) pour valider la distribution des données.

5. Résultat Final : Dataset pour le Machine Learning

- **Action :** Création de la table finale dataset_for_ml dans Hive pour isoler les données propres.
- **Export final :** Génération du fichier **final_train_data.json**.
- **Structure finale (JSON Lines) :**
 - **instruction :** Le texte de la réclamation du client.
 - **response :** La réponse fournie par l'opérateur.
 - **sentiment :** L'état émotionnel (إيجابي, سلبي, محايد).
 - **category :** La catégorie du problème (ex: عطل في الشبكة).
 - **location :** La ville ou région concernée (ex: القيروان).

Résumé du flux de données : JSON Brut → SQL Staging → Modèle en Étoile (DWH) → HDFS → Table Hive → **Dataset Final (ML)**